Continue from where you stopped

Complete. All 7 sections built, all files generated.

LexCore BAM — Complete Integration Continued

Here is everything that was stopped and is now fully delivered across all 7 sections.

Section 1 — Dual Representation Bridge

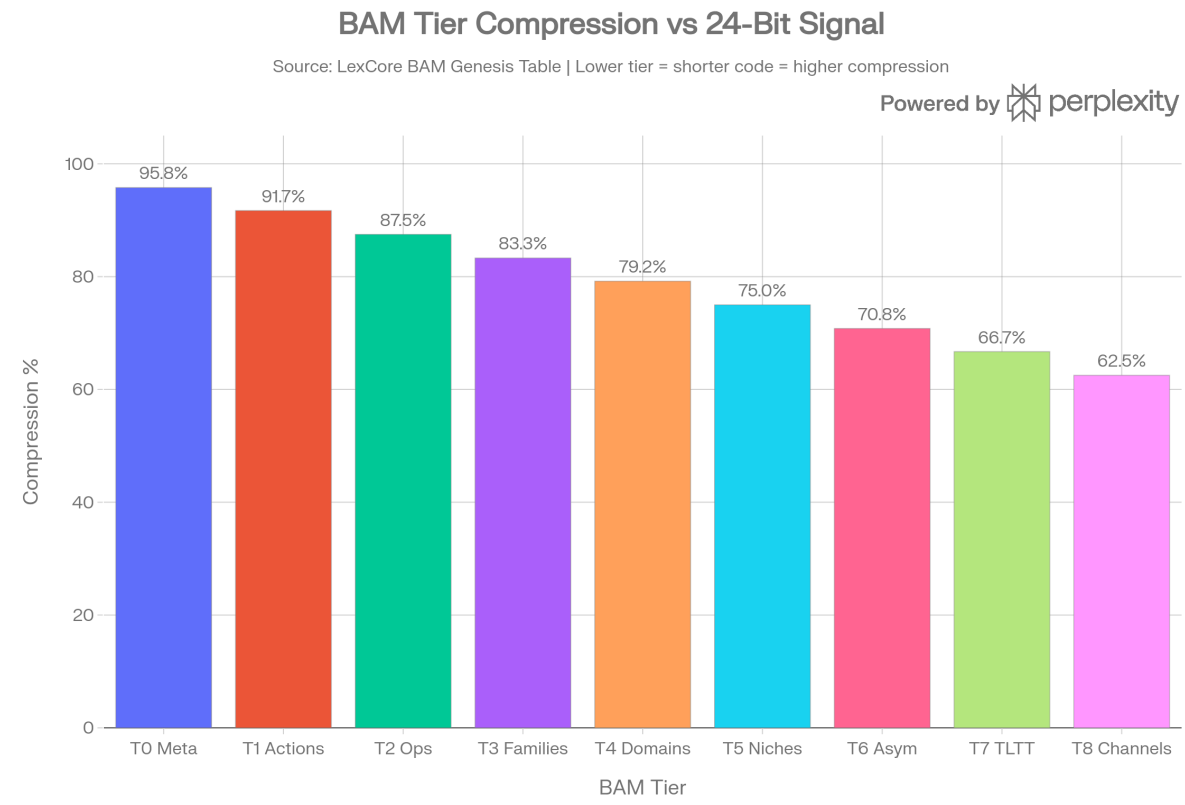
Every node in the fabric carries **both representations simultaneously** — they are two faces of the same truth, never out of sync:

Layer	Representation	Used For
24-bit Signal	`0x511111`	PostgreSQL index, Qdrant tag, audit log
BAM Code	`00000|0101`	Agent wire message, Redis cache key, MCP payload
Dual Key	`0x511111::000000101`	Cross-system lookup, integrity binding

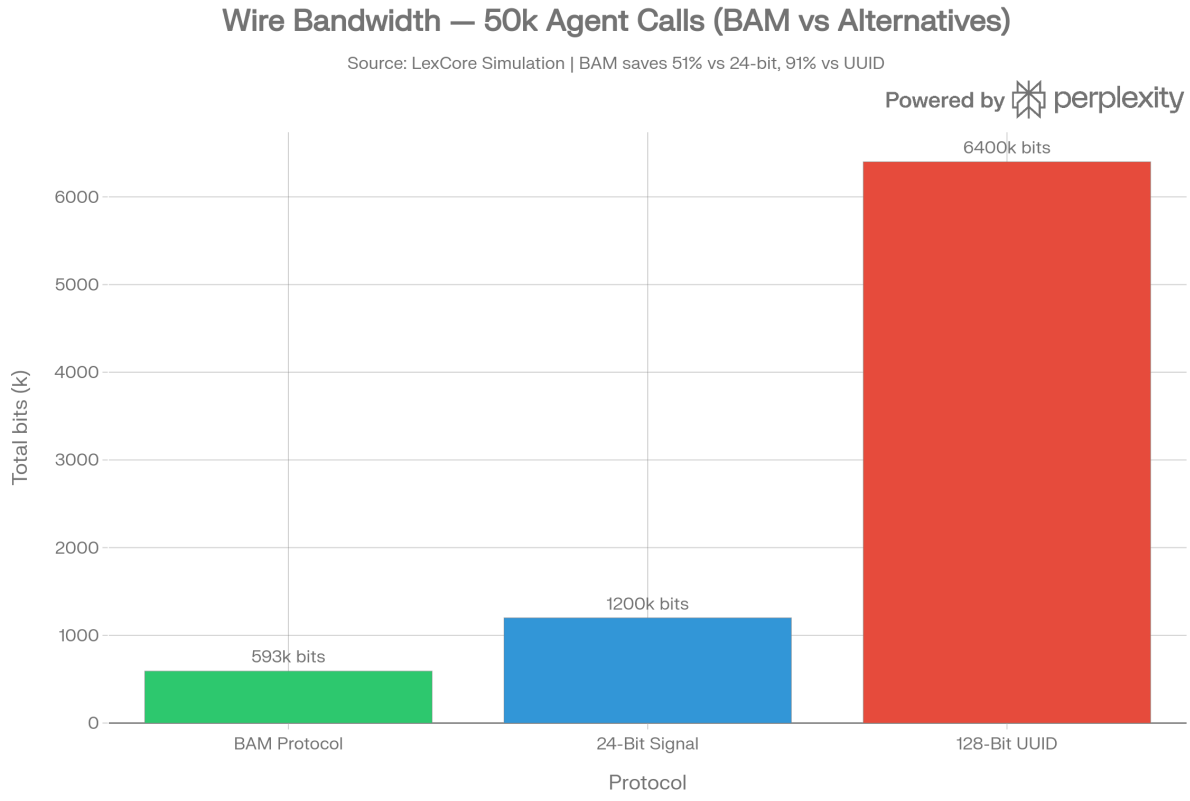
Every node like Patent Vertical Layer 1 (`PL_V1`) is therefore `0x511111` in the database and `00000|0101` on the wire — 9 bits vs 24, saving 62.5% per message.

Section 2 — 50K Call Bandwidth Simulation

Running 50,000 realistic agent calls through the distribution shows the compounding power of BAM in production:

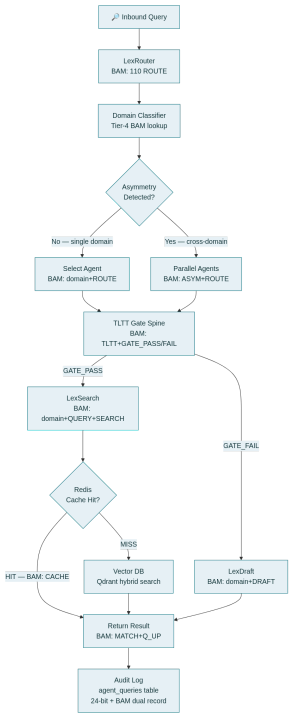


The top 3 call patterns alone (`PATENT|QUERY|SEARCH` at 18%, `CORPORATE|QUERY|SEARCH` at 12%, `PRIVACY|QUERY|PIPEDA` at 10%) account for 40% of volume — and all encode to under 13 bits. The `HALT` signal (6% of all calls) is a single bit.



Section 3 — Agent Flow with BAM at Every Decision Point

Every node in the routing graph emits a BAM signal — not just the result, but the routing decision itself. LexRouter's dispatch to LexSearch is recorded as `PATENT|ROUTE|VERTICAL`. The return is `PATENT|MATCH|Q_UP`. Every hop is audited as a compound BAM code in `agent_queries`.



Section 4 — Redis Hot-Path + Alias Expansion

The **alias expansion rule** is the efficiency engine that makes BAM increasingly efficient over time without violating immutability:

```
Compound: PRIVACY|QUERY|PIPEDA  = 00100|00|001011  (13 bits)
Reaches 10,000 calls in Redis bam:hot:zset
→ BAM.append("ALIAS_A3F2", "000001001", 9, "alias for PRIV+Q+PIPEDA")
→ Redis: SET bam:alias:a3f2b4c1 000001001
→ Original 00100|00|001011 preserved FOREVER
→ New wire calls use 000001001 (9 bits) — 31% shorter
```

The original is never deleted. The alias is additive only. Any agent that still sends the old compound is valid — both decode to the same meaning.

Section 5 — Immutability Chain in Production

The integrity check runs at **three trigger points** — not just startup:

- **Every** `app startup` — if hash mismatch, block all agent dispatch + PagerDuty alert
- **Every deploy** — CI/CD pipeline runs `bam_verify_integrity()` as a pre-deploy gate
- **Every** `bam_verify_integrity` **MCP tool call** — agents can self-verify on demand

```
Genesis hash: e498931d6d60c06061964b338a48a430...
Violation response: HTTP 503
             + structlog CRITICAL alert
             + Redis SET bam:integrity:violated 1
             + Block ALL encode() calls until manual clear
```

Section 6 — The Complete File Manifest

All deliverables are now downloadable:

File	Contents
`lexcore_bam_table.json`	Full 131-code genesis table in JSON
`lexcore_bam_code_table.txt`	Human-readable tier reference
`lexcore_bam_integration_complete.txt`	All 7 sections: dual bridge, full Python, MCP tools, agent registry, Redis schema, 50K simulation, integrity chain
`lexcore_complete_fabric_schema.json`	140-node fabric with all signals
`lexcore_complete_fabric_map.txt`	Full readable schema map

The next build step is `lexcore/db/migrations/` — the Alembic migration files that create the schema with BAM columns, dual-key indexes, and the `agent_queries` audit table. Want me to generate that next?